

The Journey of PyTorch

12 November 2024 19:14

PyTorch Overview

- Open-Source Deep Learning Library: Developed by Meta AI (formerly Facebook AI Research).
- Python & Torch: Combines Python's ease of use with the efficiency of the Torch scientific computing framework, originally built with Lua. Torch was known for high-performance tensor-based operations, especially on GPUs.

PyTorch Release Timeline

PyTorch 0.1 (2017)

- Key Features:
 - Introduced the dynamic computation graph, enabling more flexible model architectures.
 - Seamless integration with other Python libraries (e.g., numpy, scipy).
- Impact:
 - Gained popularity among researchers due to its intuitive, Pythonic interface and flexibility.
 - Quickly featured in numerous research papers.

PyTorch 1.0 (2018)

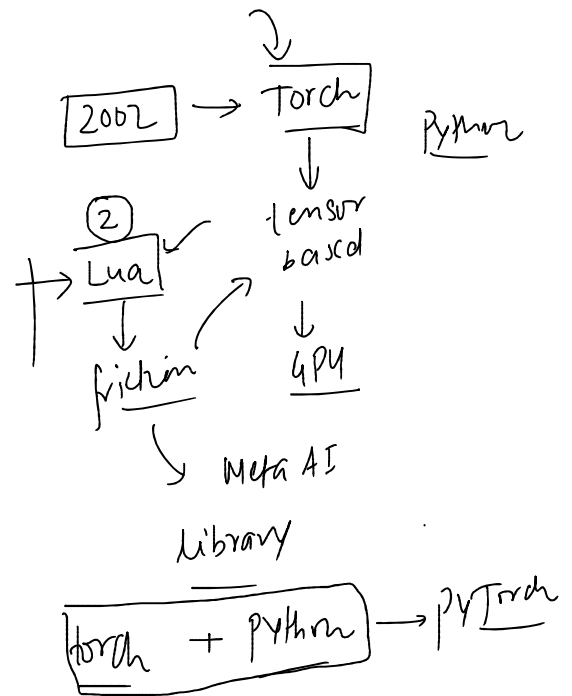
- Key Features:
 - Bridged the gap between research and production environments.
 - Introduced TorchScript for model serialization and optimization.
 - Improved performance with Caffe2 integration.
- Impact:
 - Enabled smoother transitions of models from research to deployment.

PyTorch 1.x Series

- Key Features:
 - Support for distributed training.
 - ONNX compatibility for interoperability with other frameworks.
 - Introduced quantization for model compression and efficiency.
 - Expanded ecosystem with torchvision (CV), torchtext (NLP), and torchaudio (audio).
- Impact:
 - Increased adoption by the research community and industry.
 - Inspired community libraries like PyTorch Lightning and Hugging Face Transformers.
 - Strengthened cloud support for easy deployment.

PyTorch 2.0

- Key Features:
 - Significant performance improvements.
 - Enhanced support for deployment and production-readiness.
 - Optimized for modern hardware (TPUs, custom AI chips).
- Impact:
 - Improved speed and scalability for real-world applications.
 - Better compatibility with a variety of deployment environments.



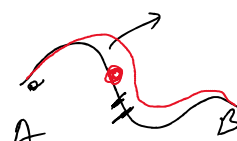
- 1) Python compatibility
- 2) dynamic computation

open-source format → use

model → weight / numbers

float → 32 bit
1 million × 32 bit
↓
32 million
↓
400mb

int
↓
8 bit
8 million
↓
100mb

A  B
static
comp

comp

Core Features

12 November 2024 19:15

1. Tensor Computations
2. GPU Acceleration
3. Dynamic Computation Graph
4. Automatic Differentiation ✓
5. Distributed Training
6. Interoperability with other libraries

PyTorch Vs TensorFlow

12 November 2024 19:15

2015 → industry



Aspect	PyTorch	TensorFlow	Verdict
Programming Language	Primarily Python; provides a Pythonic interface with deep integration.	Supports multiple languages: Python (primary), C++, Java, JavaScript (TensorFlow.js), and Swift (experimental).	Depends: PyTorch for Python-centric development; TensorFlow for multi-language support.
Ease of Use	Known for its intuitive and Pythonic syntax, making it user-friendly and easier for beginners.	TensorFlow 2.x improved usability with Keras integration, but can still be complex.	PyTorch Wins: Generally considered easier to learn and more intuitive.
Deployment and Production	Offers TorchScript for model serialization; PyTorch Mobile supports mobile deployment; growing support for production environments.	Strong production support with TensorFlow Serving, TensorFlow Lite, and TensorFlow.js; more mature tools.	TensorFlow Wins: More mature and comprehensive deployment options.
Performance	Competitive performance; <u>dynamic graphs may</u> introduce slight overhead; optimized with TorchScript and JIT compilation.	Optimized through static graphs and XLA compiler; efficient for large-scale models.	Tie: Both offer high performance; differences are often negligible in practice.
Community and Ecosystem	Rapidly growing community; strong in <u>academia</u> ; rich ecosystem with libraries like torchvision and integration with Hugging Face.	<u>Large and established community</u> ; extensive ecosystem with tools like TensorBoard and TFX; widely used in industry.	Depends: PyTorch excels in research community; TensorFlow has a broader industry ecosystem.
High-Level APIs	Uses native modules like torch.nn; high-level interfaces provided by <u>PyTorch Lightning</u> and <u>Fast.ai</u> .	Integrates Keras (<u>tf.keras</u>) as the high-level API.	TensorFlow Wins: Keras provides a more established and user-friendly high-level API.
Mobile and Embedded Deployment	PyTorch Mobile enables deployment on iOS and Android; supports model optimization like quantization.	<u>TensorFlow Lite</u> provides robust support for mobile and embedded devices; TensorFlow.js for web deployment.	TensorFlow Wins: More mature and versatile options for mobile and embedded deployment.
Preferred Domains	Favored in research and <u>academia</u> ; excels in rapid prototyping; strong in computer vision and NLP tasks.	Widely used in <u>industry</u> and <u>production</u> ; versatile across various domains.	Depends: PyTorch for research; TensorFlow for industry applications.
Learning Curve	Easier to learn due to intuitive design and dynamic execution.	Steeper learning curve; improved in TensorFlow 2.x but can still be complex.	PyTorch Wins: More beginner-friendly.
Interoperability	Seamless integration with Python libraries; supports exporting models to <u>ONNX</u> .	Interoperable through TensorFlow Hub and SavedModel; supports <u>ONNX</u> .	PyTorch Wins: Better integration with Python

<u>Interoperability</u>	Seamless integration with Python libraries; supports exporting models to <u>ONNX</u> format.	Interoperable through TensorFlow Hub and SavedModel; supports <u>ONNX</u> with some <u>limitations</u> .	PyTorch Wins: Better integration with Python ecosystem.
<u>Customizability</u>	High level of customization; easier to implement custom layers and operations.	Custom operations possible but can be complex; TensorFlow 2.x improves flexibility.	PyTorch Wins: Greater customizability and flexibility.
→ <u>Deployment Tools</u>	TorchServe for model serving; integrates with AWS, Azure, and Google Cloud.	TensorFlow Serving, <u>TensorFlow Extended (TFX)</u> for ML pipelines; strong cloud support.	TensorFlow Wins: More mature deployment tools and pipeline support.
<u>Parallelism and Distributed Training</u>	Supports distributed training with <code>torch.distributed</code> ; enhanced by libraries like Horovod.	Extensive support with <code>tf.distribute.Strategy</code> ; optimized for large-scale computing.	TensorFlow Wins: More advanced and user-friendly distributed training options.
<u>Model Zoo and Pre-trained Models</u>	Access via <u>TorchVision</u> , <u>Hugging Face</u> ; strong community sharing.	TensorFlow Hub offers a wide range; extensive community models.	Tie: Both offer extensive pre-trained models; choice depends on specific needs.

↘
 1k ass 4
models

PyTorch Core Modules

12 November 2024

19:16

Core PyTorch Modules

Module	Description
<code>torch</code>	The core module providing multidimensional arrays (tensors) and mathematical operations on them.
<code>torch.autograd</code>	Automatic differentiation engine that records operations on tensors to compute gradients for optimization.
<code>torch.nn</code>	Provides a neural networks library, including layers, activations, loss functions, and utilities to build deep learning models.
<code>torch.optim</code>	Contains optimization algorithms (optimizers) like SGD, Adam, and RMSprop used for training neural networks.
<code>torch.utils.data</code>	Utilities for data handling, including the <code>Dataset</code> and <code>DataLoader</code> classes for managing and loading datasets efficiently.
<code>torch.jit</code>	Supports Just-In-Time (JIT) compilation and TorchScript for optimizing models and enabling deployment without Python dependencies.
<code>torch.distributed</code>	Tools for distributed training across multiple GPUs and machines, facilitating parallel computation.
<code>torch.cuda</code>	Interfaces with NVIDIA CUDA to enable GPU acceleration for tensor computations and model training.
<code>torch.backends</code>	Contains settings and allows control over backend libraries like cuDNN, MKL, and others for performance tuning.
<code>torch.multiprocessing</code>	Utilities for parallelism using multiprocessing, similar to Python's <code>multiprocessing</code> module but with support for CUDA tensors.
<code>torch.quantization</code>	Tools for model quantization to reduce model size and improve inference speed, especially on edge devices.
<code>torch.onnx</code>	Supports exporting PyTorch models to the ONNX (Open Neural Network Exchange) format for interoperability with other frameworks and deployment.

PyTorch Domain Libraries

Library	Description
<u>torchvision</u>	Provides <u>datasets</u> , <u>model architectures</u> , and <u>image transformations</u> for computer vision tasks.
<u>torchtext</u>	Tools and datasets for natural language processing (<u>NLP</u>), including data preprocessing and vocabulary management.
<u>torchaudio</u>	Utilities for audio processing tasks, including I/O, transforms, and pre-trained models for speech recognition.
<u>torcharrow</u>	A library for accelerated data loading and preprocessing, especially for tabular and time series data (experimental).
<u>torchserve</u>	A PyTorch model serving library that makes it easy to <u>deploy trained models</u> at scale in production environments.
<u>pytorch_lightning</u>	A lightweight wrapper for PyTorch that simplifies the training loop and reduces boilerplate code, enabling scalable and reproducible models.

Popular PyTorch Ecosystem Libraries

Library	Description
 Hugging Face Transformers	Provides state-of-the-art pre-trained models for NLP tasks like text classification, translation, and question answering, built on PyTorch.
 Fastai	High-level library that simplifies training fast and accurate neural nets using modern best practices, built on top of PyTorch.
 PyTorch Geometric	Extension library for geometric deep learning, including graph neural networks and 3D data processing.
 TorchMetrics	A modular metrics API for PyTorch, compatible with PyTorch Lightning and provides standardized implementations of many common metrics.
TorchElastic	Enables dynamic scaling of PyTorch distributed training jobs, allowing for elasticity in resource management.
 Optuna	An automatic hyperparameter optimization software framework, integrating well with PyTorch for tuning models.

	Catalyst	Provides high-level features for training neural networks, focusing on reproducibility and fast experimentation.
	Ignite	High-level library to help with training neural networks in PyTorch, offering a lightweight engine for training and evaluating models.
→	AllenNLP	An NLP research library built on PyTorch, designed to support researchers in deep learning for NLP.
→	Skorch	A scikit-learn compatible wrapper for PyTorch that allows the use of PyTorch models with scikit-learn utilities and APIs.
→	PyTorch Forecasting	High-level library for time series forecasting, making it easy to build, train, and evaluate complex models.
→	TensorBoard for PyTorch	Allows visualization of training metrics, model graphs, and other useful data within TensorBoard for PyTorch models.

Who uses PyTorch

12 November 2024 19:16

Company	Products/Services Using PyTorch	Description of Usage
<u>Meta Platforms (Facebook)</u>	- Facebook App - Instagram - Meta AI Research Projects	Developed PyTorch and uses it extensively for computer vision, natural language processing, and AI research across its platforms.
<u>Microsoft</u>	- Azure Machine Learning - Bing Search - Office 365 Intelligent Features	Integrates PyTorch into Azure services for AI development; employs PyTorch in search relevance, productivity tools, and various AI applications.
<u>Tesla</u>	- Autopilot System - Full Self-Driving (FSD) Capability	Uses PyTorch for training deep neural networks in computer vision and perception tasks critical for autonomous driving systems.
<u>OpenAI</u>	- GPT Models - DALL-E - ChatGPT	Utilizes PyTorch for training large-scale language models and generative models in natural language processing and computer vision.
<u>Uber</u>	- Uber Ride-Hailing Platform - Uber Eats Recommendations - Pyro (Probabilistic Programming)	Employs PyTorch for demand forecasting, route optimization, and developed <u>Pyro</u> , a probabilistic programming language built on PyTorch.

funda

NN

→ Tensors

→ Computation

→ Auto Grad

Before starting

20 November 2024 17:55

1. Boring but useful
2. Knowledge of Deep learning
3. Very similar to NumPy
4. Lecture flow
5. Very close to PyTorch official documentation
6. Watch like a lecture

What are Tensors

20 November 2024 17:56

2 Data structure arrays → tensors

- ✓ Tensor is a specialized multi-dimensional array designed for mathematical and computational efficiency.

Real-World Examples

1. Scalars: 0-dimensional tensors (a single number)

- Represents a single value, often used for simple metrics or constants.
- Example:
 - Loss value: After a forward pass, the loss function computes a single scalar value indicating the difference between the predicted and actual outputs.
 - Example: 5.0 or -3.14

2. Vectors: 1-dimensional tensors (a list of numbers) (arrays) embeddings

- Represents a sequence or a collection of values.
- Example:
 - Feature vector: In natural language processing, each word in a sentence may be represented as a 1D vector using embeddings.
 - Example: $[0.12, -0.84, 0.33]$ a word embedding vector from a pre-trained model like Word2Vec or GloVe.

3. Matrices: 2-dimensional tensors (a 2D grid of numbers)

- Represents tabular or grid-like data.
- Example:
 - Grayscale images: A grayscale image can be represented as a 2D tensor, where each entry corresponds to the pixel intensity.
 - Example: $\begin{bmatrix} 0 & 255 & 128 \\ 34 & 90 & 180 \end{bmatrix}$

4. 3D Tensors: Coloured images (RGB) 3 channel R, G, B

- Adds a third dimension, often used for stacking data.
- Example:
 - RGB Images: A single RGB image is represented as a 3D tensor (width × height × channels).
 - Examples:
 - RGB Image (e.g., 256x256): Shape $[256, 256, 3]$

5. 4D Tensors: Batches of RGB images

- Adds the batch size as an additional dimension to 3D data.
- Example:
 - Batches of RGB Images: A dataset of coloured images is represented as a 4D tensor (batch size × width × height × channels).
 - Example: A batch of 32 images, each of size 128x128 with 3 colour channels (RGB), would have shape $[32, 128, 128, 3]$.

6. 5D Tensors: Video data

- Adds a time dimension for data that changes over time (e.g., video frames).
- Example:
 - Video Clips: Represented as a sequence of frames, where each frame is an RGB image.
 - Example: A batch of 10 video clips, each with 16 frames of size 64x64 and 3 channels (RGB), would have shape $[10, 16, 64, 64, 3]$.

Why Are Tensors Useful?

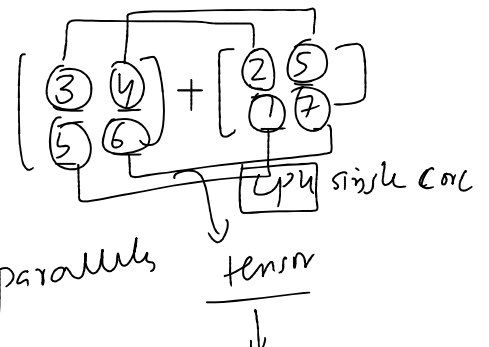
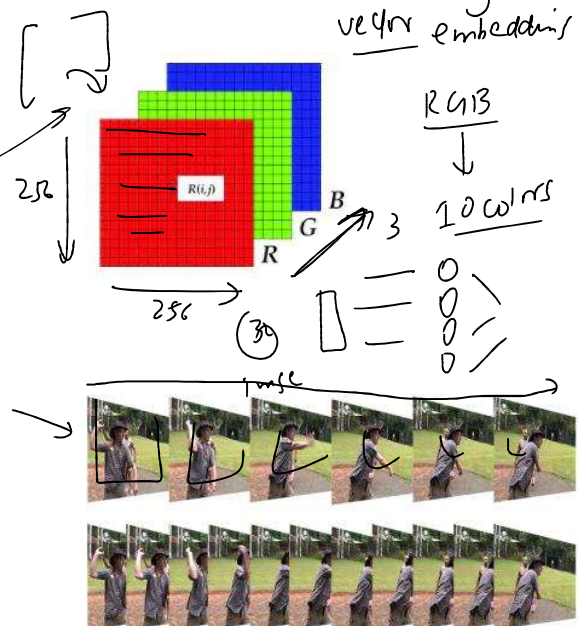
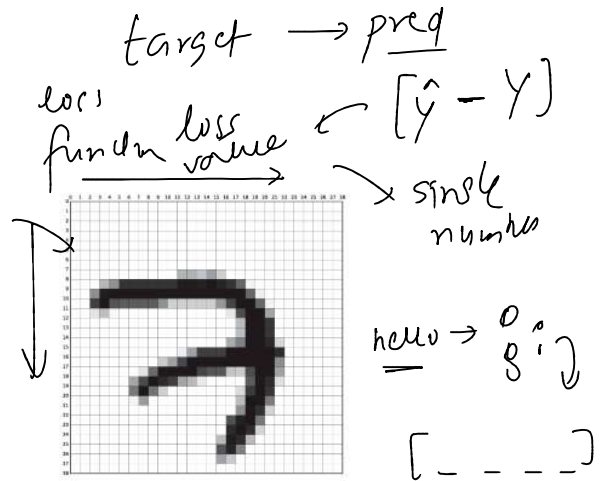
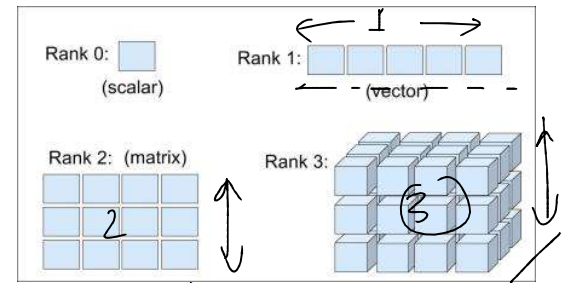
1. Mathematical Operations

- Tensors enable efficient mathematical computations (addition, multiplication, dot product, etc.) necessary for neural network operations.

2. Representation of Real-world Data

- Data like images, audio, videos, and text can be represented as tensors:
 - Images: Represented as 3D tensors (width × height × channels).
 - Text: Tokenized and represented as 2D or 3D tensors (sequence length × embedding size).

3. Efficient Computations



size).

3. Efficient Computations

- Tensors are optimized for hardware acceleration, allowing computations on GPUs or TPUs, which are crucial for training deep learning models.

Where Are Tensors Used in Deep Learning?

1. Data Storage

- Training data (images, text, etc.) is stored in tensors.

2. Weights and Biases

- The learnable parameters of a neural network (weights, biases) are stored as tensors.

3. Matrix Operations

- Neural networks involve operations like matrix multiplication, dot products, and broadcasting—all performed using tensors.

4. Training Process

- During forward passes, tensors flow through the network.
- Gradients, represented as tensors, are calculated during the backward pass.

